

Lab 5 A

Semantic Errors

Purpose:

In this lab, you will add semantic error checking to your compiler.

NOTE:

Semantic processing will be handled in two parts: Part A will run checks while we are parsing. Part B will run checks after parsing is completed. We will use the Visitor pattern for Part B.

Do NOT replace your Lab 4 test directory until instructed to do so.

Preliminaries:

There should be a function defined in `lang.y` that gets called any time a semantic error is detected. You need to add a prototype to an include file so that your AST classes can call the function when necessary. The logical place for the prototype is in `cAstNode.h`, because only AST nodes will call this function. But the prototype must be Outside the class definition because it is not a method of the class. The `cAstNode.h` supplied for Lab 4 should already have this declaration. The code the function is as follows, and it must be added to the end of `lang.y`:

```
// Function that gets called when a semantic error happens
void SemanticParseError(std::string error)
{
    std::cout << "ERROR: " << error << " near line "
                << yylineno << "\n";
    g_semanticErrorHappened = true;
    yynerrs++;
}
```

You need a definition of `g_semanticErrorHappend`. You can place the following in one of the code sections near the top of your `.y` file:

```
static bool g_semanticErrorHappened = false;
```

The semantic checks belong in the constructors and other methods you wrote in Lab 4. You should not have to change any of the code snippets in `lang.y` in order to do semantic checking with the exception of including the macros described below.

Some semantic errors mean that we need to trigger the error recovery process in the parser. Other times, we just need to note the error and continue. If you define the following two macros in your `lang.y`, you can use them to distinguish between the two cases:

```

#define CHECK_ERROR() { if (g_semanticErrorHappened) \
    { g_semanticErrorHappened = false; } }
#define PROP_ERROR() { if (g_semanticErrorHappened) \
    { g_semanticErrorHappened = false; YYERROR; } }

```

You MUST invoke one or the other after each function call that might detect a semantic error. Use PROP_ERROR if you want to trigger error recovery in the parser. For example:

```

item: this that { $$ = new Node($1,$2); PROP_ERROR(); }

```

You must write code that deals with the following semantic issues. Put the code in your AST nodes. Keep lang.y as clean as possible.

Handling TYPE_ID

In Lab 4, you added a Boolean to cSymbol. We are going to replace that mechanism with a better way to determine the type-ness of cSymbols.

1. Remove the bool from cSymbol including its getter and setter. Add a cDeclNode* instance variable to cSymbol with add a getter and setter.
2. Add the following getters to cDeclNode:

```

virtual bool IsArray() { return false; }
virtual bool IsStruct() { return false; }
virtual bool IsType() { return false; }
virtual bool IsFunc() { return false; }
virtual bool IsVar() { return false; }
virtual bool IsFloat() { return false; }
virtual bool IsInt() { return false; }
virtual bool IsChar() { return false; }
virtual int GetSize() { return 0; }
virtual cDeclNode *GetType() = 0;

```

3. Incorporate the cBaseTypeNode supplied with this lab into your directory.
4. When you create the base type symbols (char, int, etc.) create an appropriate cBaseTypeNode and set that as the decl for the cSymbol. The size of the various types is as follows:

char	1
int	4
long	8
float	4
double	8

5. Override the virtual functions declared in cDeclNode in all the decl subclasses. Note: You only need to override the ones that would return true. For example,

`cFuncDeclNode` needs to override `IsFunc()` (because it is a function), but it does not need to override `IsType()` (because it is not a type).

6. The only decl that needs to override `GetSize()` is `cBaseTypeNode`. We will add others in Lab 6.
7. Add a `GetType()` pure virtual to `cExpNode`.
8. For `cIntExprNode` the `GetType()` function needs to determine if the value is in the range of `char` (0..127) or not. If the value is in the range of a `char`, `GetType()` must do a symbol table lookup for “char”, get the decl for that symbol and return its type. Otherwise, it must lookup “int” and return the type for that.
9. Change the code in your scanner that determines if an identifier is a type. You should be able to call `GetDecl() -> IsType()` on the `cSymbol` to determine if it is a type.
10. All “things” that have (or are) declarations, need to override `GetDecl()`. Things that are decls (things that inherit from `cDeclNode`) can simply return `this`. They are their own decl. Things that have declarations (variables and functions) should return the appropriate decl, which can be obtained from their `cSymbol`.

At this point, all of your Lab 4 tests should pass. If you pass all Lab 4 tests, you are ready to set up the lab5a test directory. The files in the included tar file are intended to be expanded into a directory called “test5a”. The scripts will not run if the directory is called “test”. This directory includes Lab 4 tests as well as the Lab 5a tests. By re-running the Lab 4 tests, you will know you didn’t break anything in Lab 4.

Semantic Processing:

Your semantic processing must be such that a semantic error at some point in the source code that you are compiling should not cause a seg-fault in later processing.

You must handle the following errors.

Note: the test numbers roughly follow the numbering of this list.

1. Duplicate definitions: If an identifier is defined multiple times in the same scope, the following error should be emitted. Note: this applies to all definitions: variables, structs, arrays. We will handle functions later.

`Symbol <id> already defined in current scope`

2. Undefined references: If a variable or function is referenced that is not defined, the following error should be emitted:

`Symbol <id> not defined`

3. You must merge the information when there are multiple function declarations of the same name. For example, all of the following function declarations/definitions are legal (with all in the same scope), and the `cSymbol` returned for each must have the same ID. You are not allowed to change the ID of an existing `cSymbol` to get this behavior – you

must return the same `cSymbol`. I will allow one of two behaviors: they all return the ID of the first `foo` or they all return the ID of the last `foo`. These behaviors will have different results. When I'm testing your code, if you went with one implementation and I went with the other, I will not mark you down (the difference will be the symbol ID in the XML file).

```
int foo();  
int foo() { return 1; }  
int foo();
```

4. If a function is declared in the same scope as a previous identifier that is not a function, the following error must be emitted:

```
Symbol <id> already defined in current scope
```

5. Function declarations and definitions must match in terms of return type and number of parameters. If there are violations, one or more of the following errors should be emitted:

```
<func> previously declared with different return type  
<func> redeclared with a different number of parameters
```

6. A function is only allowed to have a single definition. You can have multiple prototypes, but if there is more than one definition for a function, the following error should be emitted:

```
<func> already has a definition
```